كيف ينمو العمل: O الكبيرة

المدة
٢٠ دقيقة

المواد
هاتف وعليه هذا الدرس بعد تنزيله، دفتر لأعداد الخطوات

أهداف التعلّم

- تعدّ الخطوات التي تأخذها الخوارزمية كلما نمت المدخلات.
- تصنّف الكود إلى O(1) وO(log n) وO(n) وO(n log n) وO(n^2).
- تستبدل طريقة O(n^2) بطريقة أسرع.

النشاط: كل زوج، أم مرور واحد؟

١. التمهيد (٣ دقائق): اكتب تخمينًا: كم تستغرق مقارنة كل زوج من مليون سجلّ على الهاتف؟
٢. العدّ (٧ دقائق): عُدّ على الورق المقارنات التي تحتاجها طريقة كل زوج لـ ٤ و٨ و١٦ عنصرًا. ماذا يحدث كلما تضاعفت n؟
٣. التحسين (٧ دقائق): اكتب نسخة المجموعة وعُدّ فحوصها للأحجام نفسها.
٤. الشرح (٣ دقائق): اكتب الطريقتين بصيغة O(n^2) وO(n)، ثم تحقّق من تخمين التمهيد على أساس عشرة ملايين خطوة في الثانية.

يكفي العدّ على الورق: يظهر النمط عند ١٦ عنصرًا.

للنقاش

- لماذا يعدّ علماء الحاسوب الخطوات بدل الثواني؟
- أين رأيت حلقة مخفيّة؟
- متى يكون O(n^2) كافيًا؟

تحقّق من نفسك

- اكتب حلقة من O(n) وأخرى من O(n^2) للقائمة نفسها.

جرّبها هذا الأسبوع

ابحث عن عيد ميلاد مشترك بين عشرة أصدقاء بطريقتين: قارن كل زوج، ثم مُرّ عليهم مرة واحدة وضع علامة على تاريخ ميلاد كلٍّ منهم في تقويم للسنة فيه خانة لكل يوم: الخانة المعلَّمة من قبل تعني عيد ميلاد مشتركًا. وعُدّ المقارنات (أو النظرات) في كل طريقة.

ورقة عمل

كيف ينمو العمل: O الكبيرة

البرمجة، المستوى ٤، الدرس ١

الاسم _____ الفصل _____ التاريخ _____

١ **يمرّ برنامج بحلقة على n من الأسماء. أيّ سطر داخل هذه الحلقة يُخفي حلقة ثانية؟**

◯ count = count + 1 ◯ if name in seen: حيث seen مجموعة ◯ if name in names: حيث names قائمة

٢ **اكتب first_repeat(values): تعيد أول قيمة تظهر مرة ثانية، أو None. ويجب أن تبقى سريعة مع ٢٠٠٠٠ قيمة، فاستعمل مجموعة لا حلقة داخل حلقة.**

```
def first_repeat(values):
    # return the first value seen twice, or None
    # (a set keeps it fast: one pass)
    return None
```

اكتب برنامجك:

٣ **ما O الكبيرة لهذه الدالة، لقائمة فيها n قيمة؟**

```
def has_pair(a, target):
    for x in a:
        for y in a:
            if x + y == target:
                return True
    return False
```

◯ $O(n^2)$ ◯ O(n) ◯ O(1) ◯ O(log n)

٤ **املأ الفراغات: يعدّ البرنامج كم مرة يمكن تنصيف ١٠٠٠ قبل أن يصل إلى 1.**

```
n = 1000
steps = 0
while n _____ 1:
    n = n _____ 2
    steps = steps + 1
print(steps)
```

بنك الكلمات: `//` `>` `-` `<`

٥ **صِل كل قطعة كود بـ O الكبيرة الخاصة بها.**

قراءة a[0]	• • • •	O(n log n)
البحث الثنائي في قائمة مرتّبة	• • • •	O(n)
حلقة واحدة على القائمة	• • • •	O(1)
الترتيب بالدمج	• • • •	$O(n^2)$
حلقة داخل حلقة على القائمة	• • • •	O(log n)

اعثر على برنامج كتبته فيه حلقة داخل حلقة. ما O الكبيرة له؟ وهل تستطيع مجموعة أو قاموس أن تزيل الحلقة الداخلية؟

تحقّق من إجاباتك

كيف ينمو العمل: O الكبيرة

١ **if name in names: حيث names قائمة**

الفحص بـ in في قائمة ينظر إلى العناصر واحدًا واحدًا، فيصنع داخل حلقة على n من الأسماء عملًا من $O(n^2)$.

٢

```
def first_repeat(values):
    seen = set()
    for v in values:
        if v in seen:
            return v
        seen.add(v)
    return None
```

مرور واحد، مع مجموعة تتذكّر ما رأيته: O(n).

٣ **$O(n^2)$**

حلقة داخل حلقة على القائمة نفسها: n × n زوجًا في أسوأ حالة، أي $O(n^2)$.

٤

```
n = 1000
steps = 0
while n > 1:
    n = n // 2
    steps = steps + 1
print(steps)
```

تنصيف ١٠٠٠ يصل إلى 1 في ٩ خطوات (500 و250 و125 و62 و31 و15 و7 و3 و1): هذا هو $\log_2 n$، نموّ البحث الثنائي.

٥ **قراءة a[0] — O(1)**

البحث الثنائي في قائمة مرتّبة — O(log n)

حلقة واحدة على القائمة — O(n)

الترتيب بالدمج — O(n log n)

حلقة داخل حلقة على القائمة — $O(n^2)$

خطوة واحدة، وتنصيف، ومرور واحد، وتقسيم ودمج، وكل زوج: أصناف النموّ الخمسة التي ستقابلها أكثر من غيرها.